

1) Специальная форма let имеет вид

$$\begin{aligned}
 &(\text{let } (x_1 e_1) \\
 &\quad (x_2 e_2) \\
 &\quad \dots \\
 &\quad (x_n e_n)) \\
 &\text{«тело»})
 \end{aligned}$$

Перед её выполнением вычисляются выражения $e_1, \dots, e_n$ в текущем окружении и создаются локальные переменные $x_1, \dots, x_n$, которые связываются со значениями выражений.

Затем выполняется тело let.

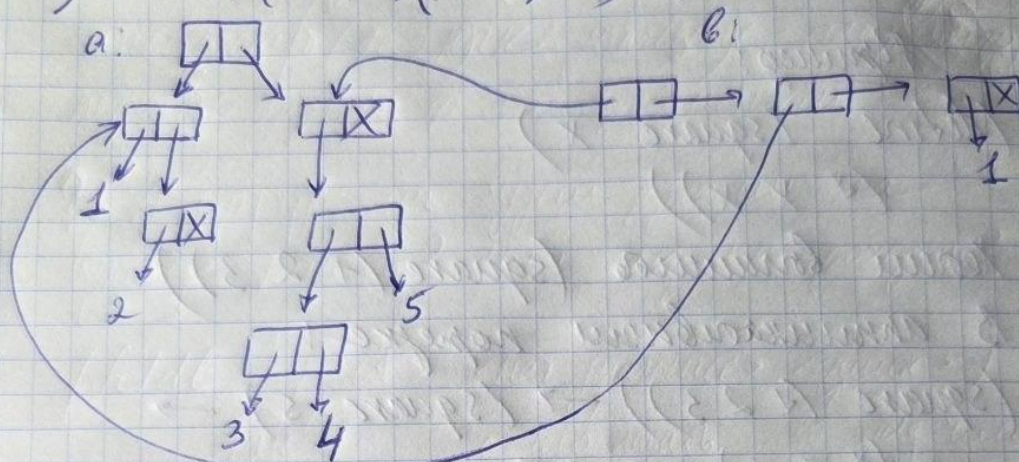
let имеет две роли, роль учета дублирования кода и повторного вычисления выражений.

Пример:

```

(define (vec-len x1 y1 x2 y2)
  (let ((x (- x2 x1))
        (y (- y2 y1)))
    (sqrt (+ (* x x) (* y y)))))
  
```

2) $a = ((1, 2), (3, 4), 5))$



$$b = ((13.4), 5) \quad (1 \ 2 \ 1)$$

3) В антикапиталистическом порядке сначала
вышли из оружейной ф-ки, а работ
тис ф-ки с новыми механизмами.

В "нормальной" порке аргументы не вычленились, а подождали так вращения.

Примечание в описании товаров
выписаны одно и то же
аргументы. В том случае и подготовка
выписывают все аргументы, даже

те, которые вероятно не пригодятся.

Пример:

$\times$ (define (square x)
(* x x))

Хотим вывести (square (+ 2 3))

В ассоциативном порядке:

(square (+ 2 3)) $\rightarrow$ (square 5) $\rightarrow$
 $\rightarrow$ (* 5 5) $\rightarrow$ 25

В "нормальном":

(square (+ 2 3)) $\rightarrow$ (* (+ 2 3) (+ 2 3)) $\rightarrow$
 $\rightarrow$ (* 5 5) $\rightarrow$ 25

(define (func x y) x)

Хотим вывести (func 1 (1 2 0))

Ассоциативный порядок выдает ошибку,

потому что вывести (1 2 0),

а в "нормальном" порядке ответ

выдет 10.